INDIAN INSTITUTE OF INFORMATION TECHNOLOGY
KALYANI

DATE:30/10/2024

ECC512 Project Report

- **Project**: Automated Parking Access Control Using Raspberry Pi and Arduino

- **Presenter's Name : SHUBHAM YADAV**
  Registration Number: 946
  Roll Number: CSE22092
  Group member: 937,943,944,945,946

Table of Contents

Project Aim

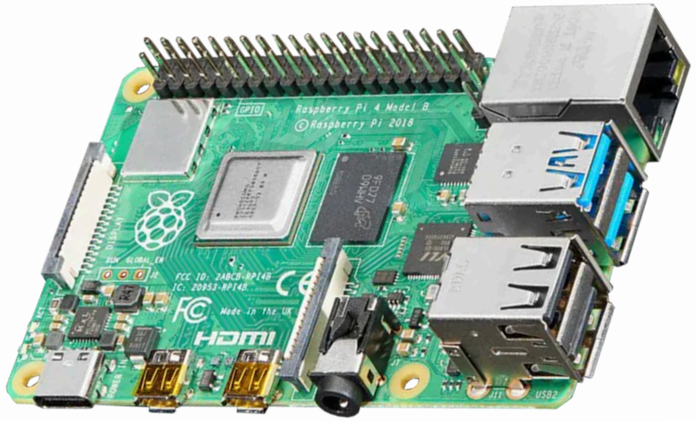
- Objective:
  - Develop an automated parking system that controls access based on vehicle status and temperature checks.
- Focus Areas:
  - Vehicle verification for restricted access
  - Gate control based on car eligibility

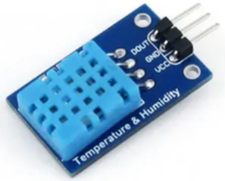
Introduction and Project Overview

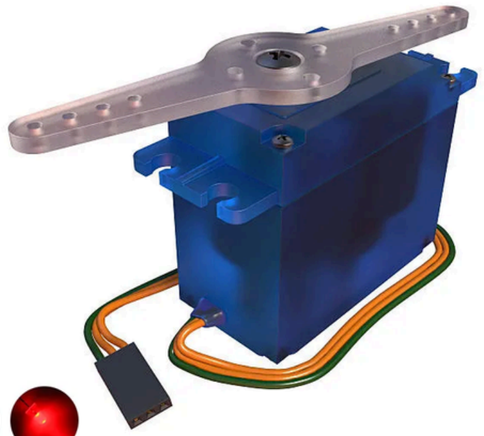
- **Project Overview:**
- **This project employs a Raspberry Pi to verify car access by checking a database of permitted vehicles.**
- **An Arduino handles environmental checks (temperature and distance) to determine gate operations.**
- **Integrated LED indicators and a buzzer provide real-time status updates for entry permissions.**
- **Key Goals:**
- **Streamline vehicle access.**
- **Ensure safety with temperature monitoring.**

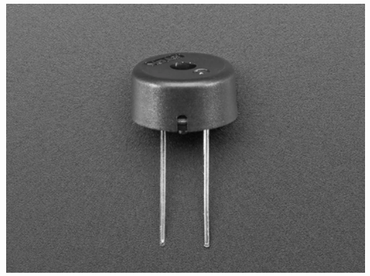
Components List

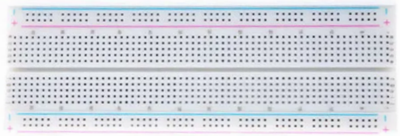
- Main Hardware Components:
- Raspberry Pi: Microprocessor for database and GPIO control
- Arduino UNO: Microcontroller for temperature and distance checks
- DHT11 Sensor: Monitors temperature
- Ultrasonic Sensor: Measures distance
- Servo Motor: Controls gate mechanism
- LEDs (Red and Green): Signal entry status
- Buzzer: Sounds alert on entry restriction
- Additional Supplies: Power supply, breadboard, jumper wires, etc.











System Design

Subsystem 1: Car Verification (Raspberry Pi)

- Process:
- Database search to verify car number
- Turns on green light for authorized access, red light with buzzer for restricted cars
- Key Functions: Database setup, car check, LED control

Subsystem 2: Gate Control (Arduino)

- Process:
- Reads temperature and distance
- Activates servo motor to open gate if conditions are met
- Key Functions: Temperature and distance measurement, gate contro
  buzzer activation

Car arrives

|

Raspberry Pi checks status

|

Permission Granted / Permission Denied

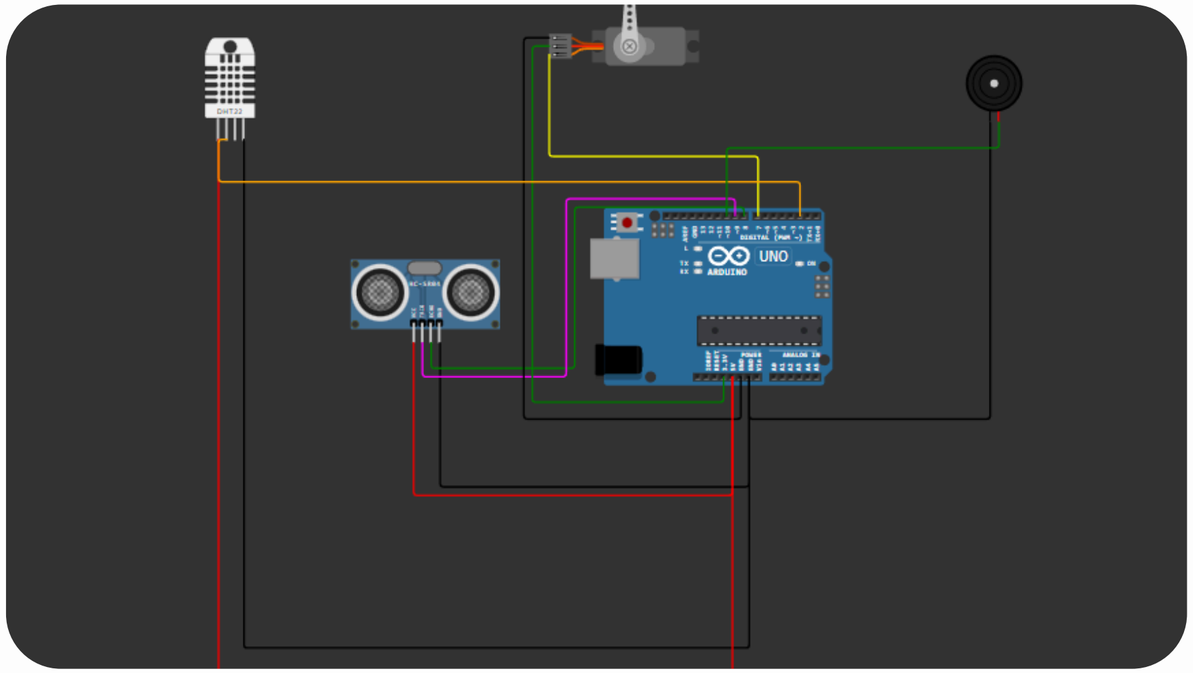
|

Arduino checks temperature and distance

|

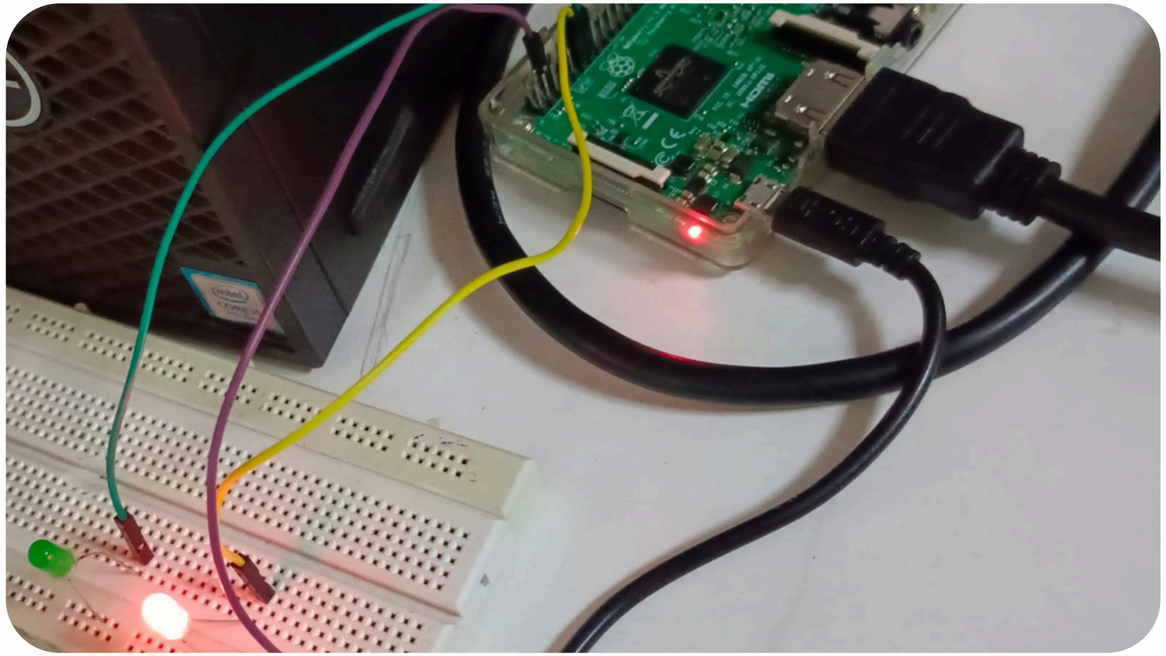
Gate opens / Gate closes

Circuit Diagram



Arduino Uno

Arduino connections to DHT11, ultrasonic sensor, and servo motor



Raspberry Pi

Raspberry Pi to LEDs and buzzer for status indication

Raspberry Pi Code Explanation

Purpose: Manages car verification, LED indicators, and buzzer control.

Main Code Segments:

1. **Database Setup and Connection: Initializes the SQLite database with car data.**
2. **Car Verification Function: Searches for the car number, controls LEDs, and outputs status.**
3. **GPIO Cleanup: Ensures safe GPIO handling after execution.**

Code File link::

car.py >>> https://drive.google.com/file/d/1NoF-28pwCdAzjDgcTaRGCLrTnmQ-ks37/view?usp=drive_link

database file>>> https://drive.google.com/file/d/1yth0zikzyPvtBncs-7RBxmE2TK_5HBGx/view?usp=drive_link

```
import sqlite3
import RPi.GPIO as GPIO
import time
from datetime import datetime

# GPIO pin configuration
RED_LED_PIN = 17
GREEN_LED_PIN = 27

# Setup GPIO
GPIO.setmode(GPIO.BCM)
GPIO.setup(RED_LED_PIN, GPIO.OUT)
GPIO.setup(GREEN_LED_PIN, GPIO.OUT)

def setup_database():
  conn = sqlite3.connect('car_database.db')
  cursor = conn.cursor()

  # Create table if it doesn't exist
  cursor.execute('''
    CREATE TABLE IF NOT EXISTS cars (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      car_number TEXT UNIQUE NOT NULL,
      owner_name TEXT NOT NULL,
      is_blacklisted BOOLEAN NOT NULL,
      car_model TEXT,
      date_of_purchase TEXT
    )
  ''')

  conn.commit()
  conn.close()

def check_car(car_number):
  conn = sqlite3.connect('car_database.db')
  cursor = conn.cursor()

  # Query the database for car details
  cursor.execute('SELECT owner_name,car_model,date_of_purchase, is_blacklisted FROM cars WHERE car_number = ?', (car_number,))
  result = cursor.fetchone()

    if is_blacklisted:
        # Turn on red light and print restriction message
        GPIO.output(RED_LED_PIN, GPIO.HIGH)
        GPIO.output(GREEN_LED_PIN, GPIO.LOW)
        print(' ')
        print(' ')
        print("Red Light - Defaulter's Car!")
        print(' ')
        print("Car information:")
        print(' ')
     # print("Car number {} belonging to {} is blacklisted.".format(car_number,owner_name))
        print("Car number: {} ".format(car_number))
        print("Car owner: {} ".format(owner_name))
        print("Blacklisted Status: {} ".format(is_blacklisted))
        print("Car model: {} ".format(car_model))
        print("Date of purchase: {} ".format(date_of_purchase))

        print("xxxxxxxxxxxxxxxxxxxxxx")
        print("Parking is restricted.")
        print("xxxxxxxxxxxxxxxxxxxxxx")
    else:
        # Turn on green light and print access message
        GPIO.output(GREEN_LED_PIN, GPIO.HIGH)
        GPIO.output(RED_LED_PIN, GPIO.LOW)
        print(' ')
        print(' ')
        print("Green Light - Access Granted")
        print(' ')
        print("Car information:")
        print(' ')
      # print("Car number {} belonging to {} is allowed to park.".format(car_number,owner_name))
        print("Car number: {} ".format(car_number))
        print("Car owner: {} ".format(owner_name))
        print("Blacklisted Status: {} ".format(is_blacklisted))
        print("Car model: {} ".format(car_model))
        print("Date of purchase: {} ".format(date_of_purchase))
        print("xxxxxxxxxxxxxxxxxxxxxx")
        print("Parking is Allowed")
        print("xxxxxxxxxxxxxxxxxxxxxx")
```

```
    else:
        print("Car not found in the database.")

    conn.close()

def add_car():
    conn = sqlite3.connect('car_database.db')
    cursor = conn.cursor()

    # Collect car details from the user
    car_number = input("Enter the car number: ")
    owner_name = input("Enter the owner's name: ")
    is_blacklisted = input("Is the car blacklisted? (yes/no): ").strip().lower() == "yes"
    car_model = input("Enter the car model: ")
    date_of_purchase = input("Enter the date of purchase (YYYY-MM-DD): ")

    # Validate date format
    try:
        datetime.strptime(date_of_purchase, "%Y-%m-%d")
    except ValueError:
        print("Invalid date format. Please use YYYY-MM-DD.")
        return

    # Insert data into database
    try:
        cursor.execute('''
            INSERT INTO cars (car_number, owner_name, is_blacklisted, car_model, date_of_purchase)
            VALUES (?, ?, ?, ?, ?)
        ''', (car_number, owner_name, is_blacklisted, car_model, date_of_purchase))
        conn.commit()
        print("Car details added successfully.")
    except sqlite3.IntegrityError:
        print("Car number already exists in the database.")

    conn.close()


# Cleanup GPIO at the end
def cleanup():
    GPIO.output(RED_LED_PIN, GPIO.LOW)
    GPIO.output(GREEN_LED_PIN, GPIO.LOW)
    GPIO.cleanup()

# Main program
if __name__ == "__main__":
    setup_database()

    try:
        while True:
            print("\nSelect an option:")
            print("1. Check Car")
            print("2. Add Car Details")
            choice = input("Enter your choice (1 or 2): ")

            if choice == '1':
                car_number = input("Enter the car number: ")
                check_car(car_number)
            elif choice == '2':
                add_car()
            else:
                print("Invalid choice. Please select 1 or 2.")

    except KeyboardInterrupt:
        print("Exiting program.")
    finally:
        cleanup()
```

Arduino Code Explanation

Purpose: Controls gate based on temperature and distance sensor data.

Key Steps in Code:

- **Temperature and Distance Check: Reads values from DHT11 and ultrasonic sensors.**
- **Conditional Logic: Allows gate access based on safe temperature and proximity.**
- **Servo and Buzzer Control: Activates motor to open/close gate; triggers buzzer if access denied.**

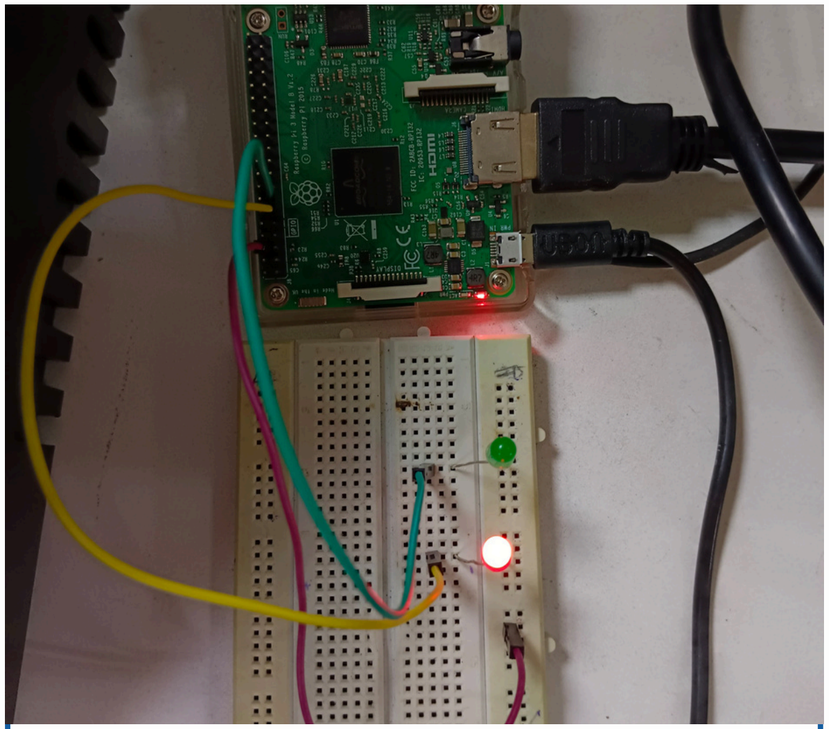
Code File link::
Arduino_code:: https://drive.google.com/file/d/1oV9dE1ZjVhX27oCFNmuGdcqboqOfJ-Bk/view?usp=drive_link

```
// Include Libraries
#include <Servo.h>
#include <DHT.h>

// Define pins for DHT11
#define DHTPIN 2   // Pin for DHT11 sensor
#define DHTTYPE DHT11 // Define DHT as DHT11

DHT dht(DHTPIN, DHTTYPE); // Initialize DHT11 sensor

// Define pins for Ultrasonic Sensor
int trigPin = 9;
int echoPin = 8;

// Define Servo Motor
Servo servo;

// Define Buzzer Pin
int buzzerPin = 10; // Pin for the buzzer

// Define Green LED Pin
int greenLEDPin = 11; // Pin for the green LED

// Variables for ultrasonic sensor and temperature
long duration;
int distance;
float temperature;


void setup()
{
 // Initialize the servo motor
 servo.attach(7);
 servo.write(0); // Close the gate initially
 delay(2000);

 // Set trigPin as OUTPUT and echoPin as INPUT
 pinMode(trigPin, OUTPUT);
 pinMode(echoPin, INPUT);

 // Set buzzerPin and greenLEDPin as OUTPUT
 pinMode(buzzerPin, OUTPUT);
 pinMode(greenLEDPin, OUTPUT); // Set green LED pin as OUTPUT

 // Initialize Serial Communication
 Serial.begin(9600);

 // Initialize DHT11 sensor
 dht.begin();

 // Add a delay to allow the DHT sensor to stabilize
 delay(2000); // Wait for 2 seconds
}

void loop()
{
 // Measure temperature from DHT11
 temperature = dht.readTemperature(); // Read temperature in Celsius

 // Handle the case where temperature reading is NaN
 if (isnan(temperature)) {
  Serial.println("Failed to read from DHT sensor!");
  return;
 }

```

```
// Clear the trigPin
digitalWrite(trigPin, LOW);
delayMicroseconds(2);

// Trigger ultrasonic pulse for 10 microseconds
digitalWrite(trigPin, HIGH);
delayMicroseconds(10);
digitalWrite(trigPin, LOW);

// Read echoPin and calculate distance in cm
duration = pulseIn(echoPin, HIGH);
distance = duration * 0.034 / 2;

// Print temperature and distance on Serial Monitor
Serial.print("Temperature: ");
Serial.print(temperature);
Serial.println(" *C");
Serial.print("Distance: ");
Serial.print(distance);
Serial.println(" cm");
```

```
// First check for temperature adequacy
if (temperature < 40) { // If temperature is adequate
 // Now check for distance adequacy
 if (distance < 50) { // If distance is also adequate
  servo.write(90); // Open the gate
  Serial.println("Gate is open. You may pass!\n");
  digitalWrite(greenLEDPin, HIGH); // Turn on the green LED
  Serial.println("Green light signal!\n");

  noTone(buzzerPin); // Turn off the buzzer
  Serial.println("Buzzer is off\n");

 }
 else { // If distance is inadequate
  servo.write(0); // Keep gate closed
  Serial.println("Gate is closed due to insufficient distance.\n");
  digitalWrite(greenLEDPin, LOW); // Ensure green LED is off
  Serial.println("No Green light signal!\n");
  noTone(buzzerPin); // Turn off the buzzer
  Serial.println("Buzzer is off\n");
 }
}
else { // If temperature is inadequate
 servo.write(0); // Keep gate closed
 Serial.println("Gate is closed due to high temperature.\n");
 digitalWrite(greenLEDPin, LOW); // Ensure green LED is off
 Serial.println(" Red light signal!\n");
 tone(buzzerPin, 1000); // Turn on buzzer at 1000 Hz
 Serial.println("Buzzer on!\n");
}

delay(2000); // Wait for 2 seconds before the next reading
}
```

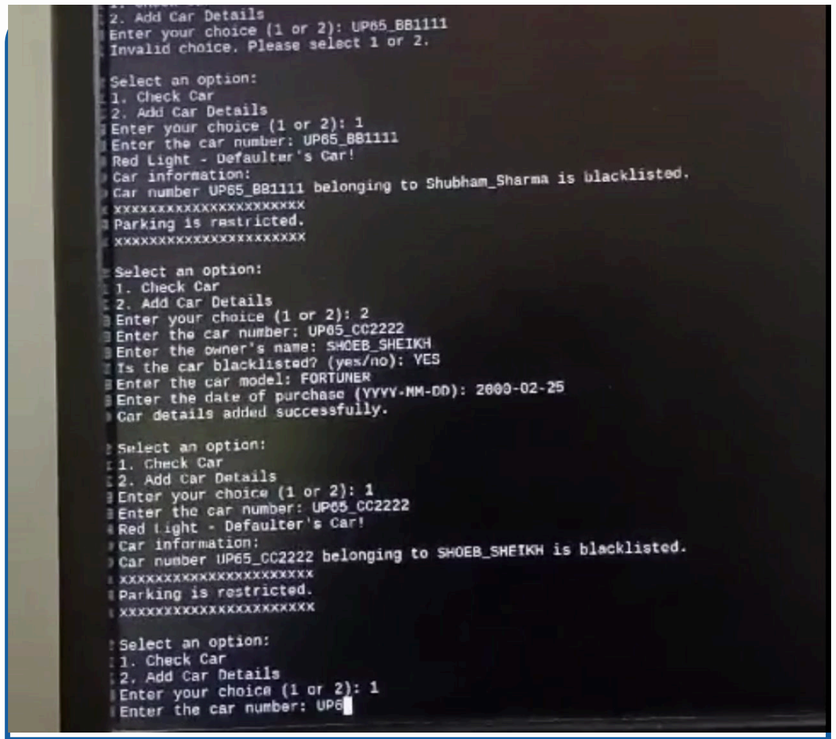
Project Photos and Video Demo: Raspberry Pi



Circuit

```
Green Light - Access Granted

Car information:

Car number: UP62_AA0000
Car owner: Shubham_Yadav
Blacklisted Status: 0
Car model: Bugatti_Chiron
Date of purchase: 2003-02-25
xxxxxxxxxxxxxxxxxxxxx
Parking is Allowed
xxxxxxxxxxxxxxxxxxxxx

Select an option:
1. Check Car
2. Add Car Details
Enter your choice (1 or 2): 1
Enter the car number: UP65_BB1111


Red Light - Defaulter's Car!

Car information:

Car number: UP65_BB1111
Car owner: Shubham_Sharma
Blacklisted Status: 1
Car model: Honda_Civic
Date of purchase: 2000-02-25
xxxxxxxxxxxxxxxxxxxxx
Parking is restricted.
xxxxxxxxxxxxxxxxxxxxx

Select an option:
1. Check Car
2. Add Car Details
Enter your choice (1 or 2):
```

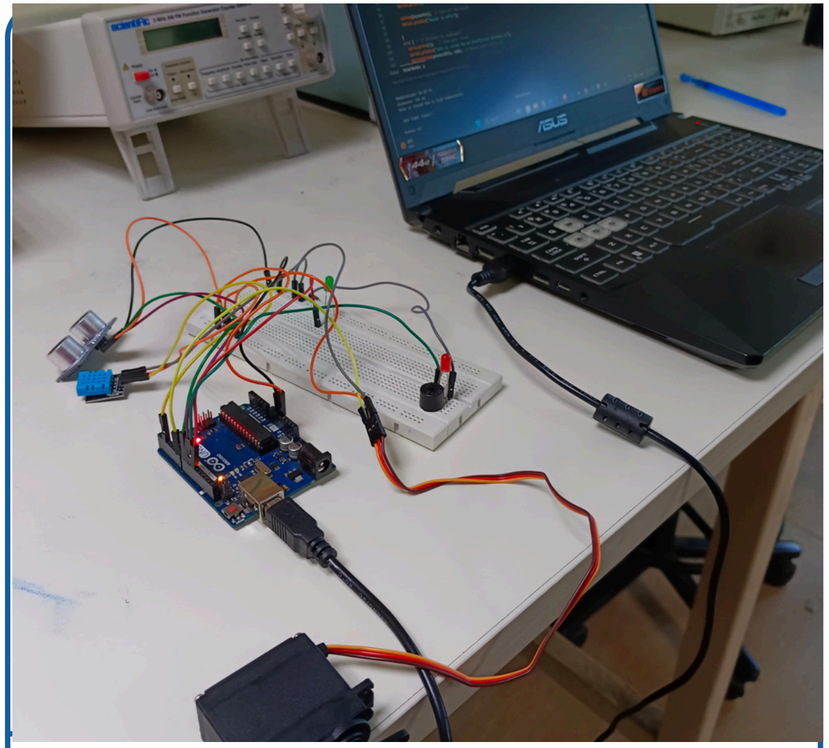
OUTPUT



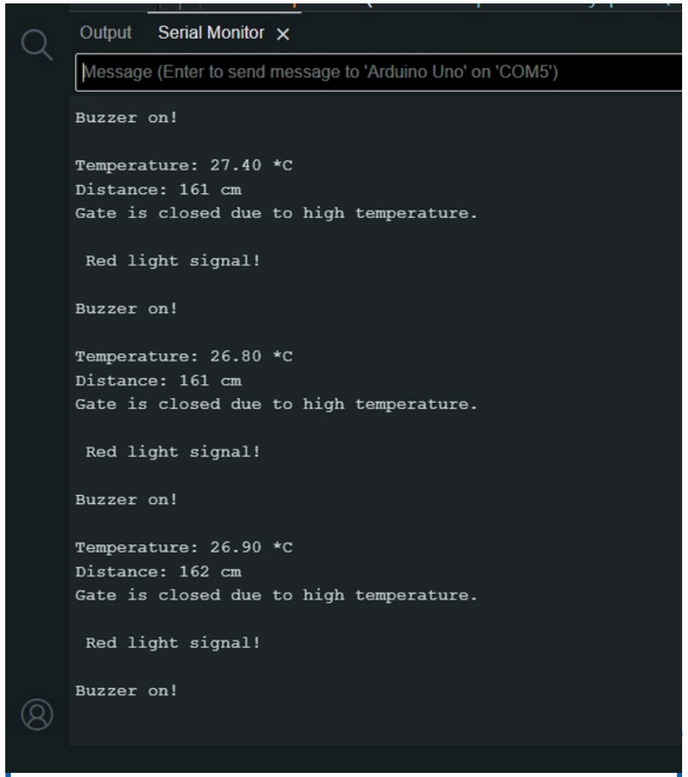
Video

video link :
https://drive.google.com/file/d/1keV0NE9FSwOnTszuki7jMQT2GW1VzJfH/view?usp=drive_link

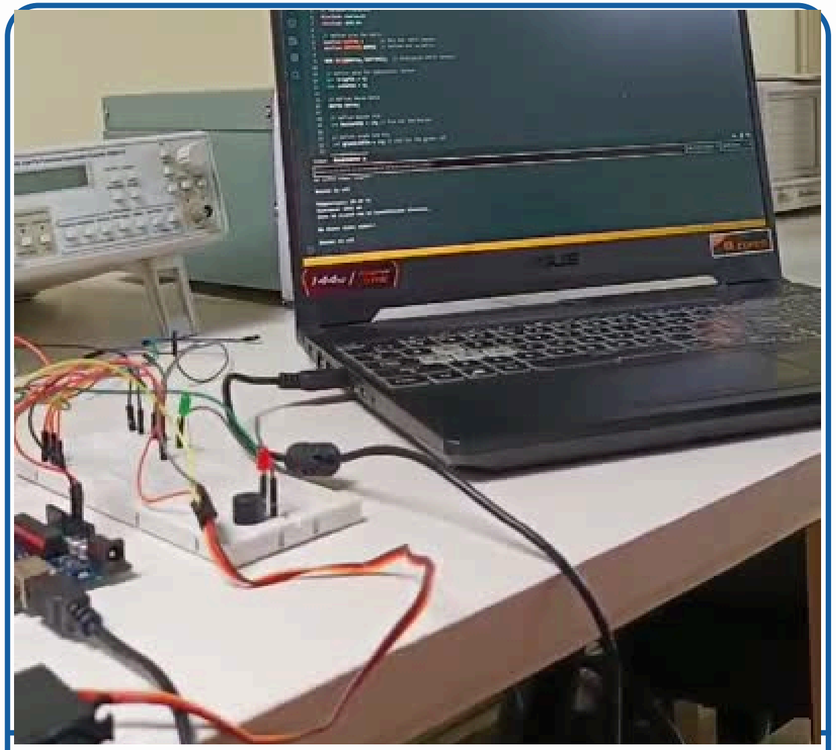
Project Photos and Video Demo: Arduino Uno



Circuit



Output



Video

video link :

https://drive.google.com/file/d/1keV0NE9FSwOnTszuki7jMQT2GW1VzJfH/view?usp=drive_link

RESULTS AND OBSERVATIONS

Results Overview:

- Authorized Cars: Green light, gate opens
- Unauthorized or High-Temperature Cars: Red light, buzzer sounds, gate remains closed

Observations:

- System effectively controls entry based on programmed conditions.
- Real-time feedback provided through LEDs and serial monitor.

Conclusion

Achievements:

- Successfully created a secure, automated parking system using microcontroller and microprocessor.

Benefits:

- Reduces need for manual verification
- Ensures safety and security with temperature-based restriction

Limitations:

- Restricted to pre-entered car numbers without real-time recognition.

Future Scope

1. **Vehicle Identification & Allocation**
   - **Recognize vehicle types (e.g., cars, bikes) and allocate designated parking zones.**
2. **Real-Time Slot Display**
   - **Display available slots dynamically for easier parking access and reduced wait times.**
3. **Traffic Flow & Entry Management**
   - **Optimize entry/exit to reduce peak-time congestion.**
4. **Remote Access & Monitoring**
   - **Provide real-time parking info via mobile app/web portal for convenience.**
5. **Predictive Analytics for Parking Patterns**
   - **Use data to predict busy times and suggest alternative spots.**

Future Scope

6. Energy Efficiency with Solar Integration
   Reduce operational costs with solar power and IoT-based sensors.
7. Advanced Security Features
   Integrate license plate recognition and facial recognition for secured access.
8. Dynamic Pricing for Revenue Optimization
   Adjust parking fees based on demand, time of day, and parking duration.
9. Broader Applications in Toll & Highway Automation
   Adapt system for toll gates, highway parking, and smart city applications.
10. AI & ML Integration
    Use AI for enhanced vehicle recognition and autonomous vehicle integration.

THANK YOU